

第2章 运算符与表达式

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 `/`、`//`、`%` 三兄弟的精确语义,尤其是**负数取模向下取整**与 C 的差异
- 理解 Python **没有** `++` / `--` 的原因,并写出等价写法
- 掌握 `and` / `or` / `not` 的短路求值与"返回操作数"特性,能写出惯用的默认值表达式
- 掌握链式比较、`==` vs `is` 的本质区别、浮点比较的正确姿势
- 掌握解构赋值、多变量交换、`:=` walrus 操作符(3.8+)
- 熟记优先级与结合性,能读懂任何复杂表达式,并会主动用括号消歧义
- 会用位运算处理标志位与掩码,知道何时该用(权限系统)何时不该用(可读性优先)

💡 从 C 到 Python:本章主题如何迁移

C 的运算符你全会——Python 保留了绝大部分,但语义上有几处**本质性差异**,每一处都是 C 程序员的第一天翻车现场。第一, `/` 永远是真除法: `7 / 2` 在 C 里是 3,在 Python 里是 `3.5`,C 的整数除法语义被拆给了 `//`。第二,取模符号不同: `-7 % 2` 在 C 里是 -1,在 Python 里是 1。第三, `++` / `--` 不存在——设计者认为 `x++` 隐藏了"既读又写"的复杂性,而且 `++x` 会被解析成 `+(+x)`,这是新手最常见的槽点。

第四,逻辑运算符不返回布尔,返回操作数本身: `"a" or "b"` 得到 `"a"` ——这是生产里最常用的技巧(默认值、链式查找)。第五, `==` 比较"值", `is` 比较"是不是同一个对象"——C 程序员容易把两者用反。

好消息:Python 把 C 的未定义行为全部消灭了——没有求值顺序 UB、没有符号溢出、没有序列点纠缠,表达式严格从左到右求值,任何疑问都有明确答案。读这一章带着一个问题: **"同样的符号,Python 和 C 的行为哪里不一样?"** ——全章所有坑都藏在答案里。

📦 前置知识自查(你已会的 C 基础)

- C 运算符的优先级与结合性(尤其 `*` 与 `++` 的纠缠、移位优先级陷阱)
- 短路求值: `&&` 与 `||` 从左到右、提前停止
- 整数除法与取模的向零截断; `%` 对负数时的符号规则
- 位运算:与/或/异或/非/左移/右移;掩码与标志位的用法
- `?:` 三目运算符; `++i` 与 `i++` 的区别

🚀 本章学习路线

1. **第一阶段(约 1 小时,算术与取模)**:读 §2.1–§2.2,在 REPL 里把负数除法、取模全部算一遍,建立新直觉
2. **第二阶段(约 1 小时,位运算与逻辑)**:读 §2.3–§2.4,重点练习 `and` / `or` 的返回值技巧
3. **第三阶段(约 1.5 小时,比较与赋值)**:读 §2.5–§2.7,做浮点比较与解构练习
4. **第四阶段(实验,约 1 小时)**:完成章末实验"表达式求值计算器",它是全章运算符的综合训练场
5. **验收标准**:能脱口而出 `-7 // 2`、`-7 % 2`、`0.1 + 0.2 == 0.3` 的答案与原因

本章知识导图

- §2.1 算术运算:/ 真除法、// 整除、% 取模、** 幂
- §2.2 自增自减的缺失与替代写法
- §2.3 位运算:全套运算符、标志位与掩码
- §2.4 逻辑运算:短路求值与"返回操作数"
- §2.5 比较运算:链式比较、== vs is、浮点比较
- §2.6 赋值:复合赋值、解构、walrus :=
- §2.7 条件表达式(三目的 Python 版)
- §2.8 优先级与结合性、特色运算符总表

§2.1 算术运算

Python 的算术运算符与 C 同名的只有 `+` `-` `*`,其余三个都有讲究。先看总表:

C 运算符	Python 运算符	差异要点
/ (整数除法截断)	/ (真除法,永远得 float)	<code>7/2 == 3.5</code> ;C 要 <code>7.0/2</code> 才行
—	// (向下取整除法)	<code>-7//2 == -4</code> ,向负无穷取整
% (余数符号同被除数)	% (模,符号同除数)	<code>-7%2 == 1</code> ,恒有 <code>a == (a//b)*b + a%b</code>
—	** (幂)	<code>2**10 == 1024</code> ;C 要 <code>pow()</code>
int 溢出(UB / 回绕)	大整数,永不溢出	<code>2**200</code> 完全精确

2.1.1 除法三兄弟与 divmod

三个除法运算符必须背熟: `/` 真除法(结果为 float)、`//` 向下取整(结果为整)、`%` 模运算(符号随除数)。Python 保证 `a == (a // b) * b + a % b` 恒成立——这是与 C"向零截断"的本质分歧:Python 选"向负无穷取整 + 模恒非负",因为这样 `%` 才能实现"循环索引"等需求(`(-1) % 7 == 6`,把 -1 安全映射到一周第 6 天)。

示例 2-1 除法三兄弟

```
print(7 / 2)      # 3.5    真除法:永远返回 float
print(7 // 2)     # 3      向下取整
print(-7 // 2)    # -4     C 给 -3,Python 向负无穷取整!
print(7 % 2)      # 1
print(-7 % 2)     # 1      C 给 -1,Python 符号随除数(2)
print(7 % -2)     # -1     除数 -2,余数符号随除数
print(-7 % -2)    # -1

# 恒等式:整除 + 模 可以互相推导
a, b = -7, 2
print(a == (a // b) * b + a % b)  # True

# divmod:一次拿到 (商, 余)
q, r = divmod(17, 5)
print(q, r)        # 3 2
```

2.1.2 幂运算与浮点警告

`2 ** 10` 是幂; `2 ** 0.5` 是开方; 注意 `-2 ** 2` 是 `-(2**2)` 得 -4, 因为 `**` 优先级高于一元负号——教科书级坑。

浮点算术必须牢记: `0.1 + 0.2` 得 `0.30000000000000004`。这不是 Python 的 bug, 是 IEEE 754 与十进制小数的固有误差, C 里一模一样。生产对策: 金额用 `decimal.Decimal`, 科学计算用容差比较, 普通场景接受误差。

示例 2-2 幂与浮点

```
print(2 ** 10)          # 1024
print(2 ** 0.5)         # 1.4142135623730951(开方)
print(-2 ** 2)          # -4 !! (先算 2**2 再取负)
print((-2) ** 2)        # 4

# 浮点误差: 人尽皆知的 0.1+0.2
print(0.1 + 0.2)        # 0.30000000000000004
print(0.1 + 0.2 == 0.3) # False

# 金额计算用 Decimal
from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2")) # 0.3 精确
```

⚠ 误区 1: 把 `//` 当作"C 的整数除法"直接搬

`7 // 2 == 3` 和 C 一致, 但 `-7 // 2 == -4` 与 C 的 -3 不同。C 的整除向零截断, Python 的 `//` 向负无穷取整。生产场景: 计算分页页码、环形缓冲索引时, `(-1) // 7` 在 Python 里是 -1, 若按 C 的直觉写成 `(-1) / 7` 再取整会差 1。只要涉及负数, 先想清楚再写。

§2.2 没有 ++ 与 --

Python 没有自增自减运算符。设计者认为 `i++` 既是有值的表达式又是带副作用的语句, 这种"双重身份"是 C 求值顺序 UB 与无数 bug 的源头。Python 的立场: 修改变量就是赋值语句, 写 `i += 1`。注意 `++i` 在 Python 里**不报错**——它被解析成 `+(+i)` (正号运算符), C 程序员写顺手了会得到"看起来没错其实没自增"的代码, 这是本章第二著名的坑。

示例 2-3 自增的替代写法

```
i = 0
i += 1          # 唯一正确的自增写法
print(i)        # 1

# 危险: ++i 不报错, 但只是两个正号!
j = 5
print(++j)      # 5 (不是 6, 也不是语法错误)

# C 的 for(i=0; i<n; i++) 在 Python 里:
n = 10
for i in range(n):
    pass
i = 0
while i < n:    # 或等价 while
    i += 1
```

§2.3 位运算

Python 的位运算符与 C 完全同名: `& | ^ ~ << >>`。差异只有一点:右移是**算术右移**(负数高位补 1),因为 int 任意精度,"符号位"天然存在, `-8 >> 1 == -4`,与 C 的带符号整数一致。

位运算生产用途:①标志位组合(权限系统);②掩码提取(协议解析)。不推荐用位运算做快速乘除 2 的幂——写 `x // 16` 比 `x >> 4` 好读,解释器里性能差异可忽略。

示例 2-4 位运算与权限标志

```
# 权限标志:每一位代表一种权限
READ = 1          # 0b001
WRITE = 2          # 0b010
EXEC = 4           # 0b100

perm = READ | EXEC    # 组合 → 5
print(bin(perm))      # 0b101
print(perm & READ)     # 1(非 0 即有该权限)
print(bool(perm & WRITE)) # False(无写权限)

# 掩码提取:从协议字段中取比特
packet = 0b11010100
low4 = packet & 0b1111    # 取低 4 位
high4 = packet >> 4       # 取高 4 位
print(low4, high4)        # 4 13

# 负数右移是算术右移,保持符号
print(-8 >> 1)           # -4
print(-8 << 1)           # -16

# 位运算查权限:非 0 即拥有(与 C 的 if (perm & FLAG) 同构)
print(bool(perm & WRITE)) # False
```

§2.4 逻辑运算:短路与返回值

Python 的逻辑运算符是 `and / or / not` (不是 `&& / || / !`),与 C 的对应关系如下:

C 的写法	Python 的写法	差异
<code>a && b</code>	<code>a and b</code>	C 返回 0/1;Python 返回操作数本身
<code>a b</code>	<code>a or b</code>	同上,短路规则一致
<code>!x</code>	<code>not x</code>	not 返回布尔 True/False
<code>x>=0 && x<100</code>	<code>0 <= x < 100</code>	链式比较一步到位

语义有三条,第二条是 C 程序员必须重构直觉的:

1. **短路求值**与 C 完全一致:从左到右,能定则停; `False and 任何` 不再求右边
2. **返回值是操作数本身**,不是布尔! `a and b` 返回"使表达式定值的那一侧"的值: `a` 为假返回 `a`,否则返回 `b`; `a or b` 为真返回 `a`,否则返回 `b`
3. 真值判定:非零数、非空容器、`True` 为真;0、空字符串/列表/字典、`None`、`False` 为假

示例 2-5 短路与“返回操作数”

```
# 短路:右侧根本不会执行
def side_effect():
    print("被调用了")
    return True

print(False and side_effect())    # False,不打印(短路)
print(True or side_effect())      # True,不打印(短路)
print(True and side_effect())     # 打印,返回 True

# 返回操作数:默认值写法
name = "" or "匿名用户"          # "" 为假 → 取 "匿名用户"
count = 0 or 10                  # 0 为假 → 10
print(name, count)               # 匿名用户 10

# 链式取第一个非空值(等价于 C 里一串 if)
a, b, c = "", None, "最终值"
print(a or b or c)               # 最终值

# 注意:or 不能用来判断"是否为 None"(0 和 "" 也被吃掉)
level = 0
print(level or 5)                # 5 — 但 0 可能合法!
print(level if level is not None else 5)  # 0:显式判 None
```

⚠ 误区 2: or 默认值把合法假值吃掉了

`x or 默认` 在 `x` 是 0、空串、空列表时也返回默认值。若这些“假值”业务上合法(分数 0、空名单),就用 `x if x is not None else 默认`。真实事故:统计系统用 `avg or 0` 处理“无数据”,平均分为 0 的班级全部被污染——区分“没有值(None)”与“值为假”是生产必修课。

§2.5 比较运算

比较运算符 `==` `!=` `<` `>` `<=` `>=` 都在,但有两个 Python 特性:一是**链式比较** `0 <= x < 100` 等价于 C 的 `x >= 0 && x < 100`;二是 `==` 与 `is` 的分工: `==` 比较“值相等”(递归比较容器内容), `is` 比较“同一个对象”。判 `None` 必须用 `is None` (PEP 8 强约定):因为 `None` 是单例,“是不是 `None`”本质是身份问题,用 `==` 在自定义类型上可能被 `__eq__` 欺骗。

示例 2-6 链式比较与 `==` vs `is`

```

# 链式比较:C 要写两遍变量,Python 一次搞定
x = 42
print(0 <= x < 100)          # True
print(100 > x >= 40)          # True(反向链式也行)

# == 比较值,is 比较身份
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)                 # True:内容逐元素相等
print(a is b)                 # False:两个不同对象

# 小整数缓存陷阱:解释器对 -5~256 做了缓存
p, q = 256, 256
print(p is q)                 # True(缓存!)
m, n = 257, 257
print(m is n)                 # 可能是 False(未缓存)
# 结论:比数值用 ==,is 只用于 None 与单例

value = None
print(value is None)          # True(生产标准写法)

```

2.5.1 浮点比较的正确姿势

`0.1 + 0.2 == 0.3` 是 `False`,所以浮点"相等"要用容差。 `math.isclose` 处理相对误差与绝对误差,生产直接用它。

示例 2-7 浮点比较

```

import math

print(0.1 + 0.2 == 0.3)          # False(二进制浮点误差)
print(math.isclose(0.1 + 0.2, 0.3)) # True(默认容差 1e-09)

# 小数值场景要显式指定绝对容差
print(math.isclose(1e-10, 0.0, abs_tol=1e-9)) # True

# 逐元素比较
scores = [99.9, 100.0, 98.5]
hits = sum(1 for s in scores if math.isclose(s, 99.9, rel_tol=1e-6))
print(hits)                       # 1

```

§2.6 赋值:复合赋值、解构与 walrus

2.6.1 复合赋值与多变量解构

`+=` `-=` `*=` `/=` `//=` `%=` `**=` `&=` `|=` `^=` `<<=` `>>=` 全都在,语义与 C 一致(先求右侧再重新绑定)。Python 用**元组解构**实现更强的多变量赋值:

C 的写法	Python 的写法
<code>tmp=a; a=b; b=tmp;</code>	<code>a, b = b, a</code>
<code>int x, y; x=1; y=2;</code>	<code>x, y = 1, 2</code>
<code>sscanf(s, "%d %d", &x, &y)</code>	<code>x, y = map(int, s.split())</code>
<code>a[0]=a[1]=a[2]=0;</code>	<code>a[:3] = [0, 0, 0] (切片赋值)</code>

示例 2-8 解构赋值

```
# 多变量同时赋值
a, b, c = 1, 2, 3
print(a, b, c)           # 1 2 3

# 经典交换(不需要临时变量)
x, y = 10, 20
x, y = y, x
print(x, y)              # 20 10

# 从列表/元组/字符串解构
name, age = ("张三", 18)    # 等价于 name, age = "张三", 18
first, *rest = [1, 2, 3, 4] # *rest 收走其余
print(first, rest)         # 1 [2, 3, 4]
head, *middle, tail = "hello"
print(head, middle, tail)  # h ['e', 'l', 'l'] o

# 个数不匹配会报错
# a, b = [1, 2, 3]          # ValueError: too many values to unpack
```

2.6.2 walrus 操作符 := (3.8+)

`:=` 叫“海象操作符”,在**表达式内部**完成赋值,返回被赋的值。最典型的用途:避免“先调用再判断”写两遍;在推导式里复用计算结果。

示例 2-9 walrus 操作符

```
import re

# 场景 1:正则匹配,只写一次调用
text = "订单号:ORD-2026-0001"
if (m := re.search(r"ORD-\d+", text)):
    print("找到:", m.group(0))    # 找到: ORD-2026-0001

# 场景 2:逐行读取直到空(while 里最常用)
with open("data.txt", encoding="utf-8") as fh:
    while (line := fh.readline()):
        print(line.strip()[:20])

# 场景 3:推导式内复用计算结果
nums = [1, 4, 9, 16, 25]
squares = [y for x in nums if (y := x ** 0.5) > 3]
print(squares)                  # [4.0, 5.0]
```

工程建议:walrus 是好工具,别当玩具

`:=` 的价值是“消除重复计算”,过度使用会伤可读性。判据:去掉它后必须写两遍同一个昂贵调用(正则匹配、读文件、查库),就用;只为少写一行,别用。`while (line := fh.readline())` 是官方惯用法,放心用。

§2.7 条件表达式(三目的替代)

C 的 `?:` 在 Python 里写作 真值分支 `if 条件 else 假值分支`——顺序反直觉,但只此一种。它是**表达式**(有值),可嵌在任何需要值的地方;没有 C 的“语句版三目”。Python 的 `if` 是语句不能当表达式用,函数内“按条件返回不同值”时,条件表达式是首选。

示例 2-10 条件表达式

```
age = 20
stage = "成年" if age >= 18 else "未成年"
print(stage)                # 成年

# 嵌套:不推荐(可读性差),最多两层
score = 85
level = "优" if score >= 90 else ("良" if score >= 80 else "及格")
print(level)                # 良

# 在列表推导里当过滤器用
nums = [i if i % 2 == 0 else -i for i in range(6)]
print(nums)                 # [0, -1, 2, -3, 4, -5]

# 生产更常用:函数内提前返回代替嵌套
def desc(age):
    if age < 0:
        return "非法"
    return "成年人" if age >= 18 else "未成年人" # 返回条件表达式
```

§2.8 优先级与特色运算符

Python 的优先级从高到低(常用部分): `**` → 一元 `+` `-` `~` → `*` `/` `//` `%` → `+` `-` → `<<` `>>` → `&` → `^` → `|` → 比较 → `not` → `and` → `or` → 赋值/条件表达式。与 C 的关键差异: `**` 高于一元负号(前面已见),且**没有逗号表达式**、没有指针运算符、没有 `sizeof` 陷阱。

Python 独有运算符: `in` (成员判断)、`is` (身份判断)、`:=` (赋值表达式)、`@` (矩阵乘,科学计算用;也用于装饰器,见第12章)。

示例 2-11 优先级实战


```
# and/or 低于比较:下面的写法与直觉一致
x = 5
print(x > 0 and x < 10)      # True(先比较,再 and)

# 但布尔运算混合时要小心
a, b, c = 1, 0, 2
print(a or b and c)          # 1(and 优先于 or,先算 b and c)
print((a or b) and c)        # 2(加括号改变语义)

# in 成员判断:list 是线性查找,set/dict 是哈希查找
names = ["张三", "李四"]
print("张三" in names)       # True
big = {i: i for i in range(10000)}
print(9999 in big)           # True(dict 查键,O(1))

# 生产建议:复杂表达式一律加括号,别赌优先级
# 反例:if a and b or c and d  —  没人能一眼读懂
# 正例:if (a and b) or (c and d)
```

⚠ 误区 3:把 in 当 strstr 用,忘了它的复杂度

`x in list` 是 $O(n)$ 线性查找;在大列表里做十万次 `in` 就是 $O(n \cdot m)$ 灾难。生产做法:查找频繁的集合用 `set` 或 `dict` (哈希, $O(1)$ 平均)。改一行数据结构,性能提升几个数量级——这是 Python 优化里性价比最高的一招,务必养成"凡是要查重/查找,先想 `set`"的肌肉记忆。

本章速查表

主题	写法 / 结论
真除法	<code>a / b</code> 恒为 <code>float(7/2 == 3.5)</code>
整除	<code>a // b</code> 向下取整(<code>-7//2 == -4</code>)
取模	<code>a % b</code> 符号随除数(<code>-7%2 == 1</code>)
自增	没有 <code>++</code> ; <code>i += 1</code> ; <code>++i</code> 是正号不报错
位运算	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code> 同名同义;右移为算术右移
逻辑运算	<code>and</code> / <code>or</code> / <code>not</code> ;短路;返回操作数
默认值	<code>a or b</code> ;判 None 用 <code>x if x is not None else b</code>
链式比较	<code>0 <= x < 100</code> 等价 C 的 <code>x>=0 && x<100</code>
相等判断	<code>==</code> 比值; <code>is</code> 比身份; None 用 <code>is None</code>
浮点比较	用 <code>math.isclose(a, b)</code> 而非 <code>==</code>
walrus	<code>x, y = y, x</code> ; <code>while (line := f.readline())</code> (3.8+)
三目	<code>a if 条件 else b</code> (注意顺序!)
优先级	<code>**</code> > 一元 > 乘除 > 加减 > 移位 > 位与 > 比较 > <code>not</code> > <code>and</code> > <code>or</code>
成员判断	<code>x in 容器</code> ;set/dict 是哈希查找, $O(1)$

最小必会清单

- 能脱口而出 `7/2`、`7//2`、`-7//2`、`-7%2` 四个值
- 能解释为什么 `-2 ** 2` 是 -4,以及怎么写才是 4
- 能写出 `++i` 在 Python 里的真实含义与正确写法
- 能利用 `a or b` 写默认值,并说清与 `if a is None` 的适用边界
- 能说出 `==` 与 `is` 的区别,以及小整数缓存现象
- 能写出链式比较 `0 <= x < 100` 与 C 的等价形式
- 能用 `math.isclose` 做浮点相等比较
- 能写出 `x, y = y, x` 与 `first, *rest = lst`
- 能解释 walrus 解决什么问题,并写一个 `while (line := ...) 示例`
- 能写出三目表达式并注意顺序;知道查重用 `set` 而非 `list`

自测题

Q 1. 计算下列各式的值(先不运行,手算): `10 / 4`、`10 // 4`、`-10 // 4`、`-10 % 4`、`10 % -4`。

✓ 参考答案

`2.5`、`2`、`-3`、`2`、`-2`。验证:前四个满足 `-10 == (-3)*4 + 2`;第五个 `10 == (-3)*(-4) + (-2)`。模的符号永远与除数一致。

Q 2. `a = 3`; `b = 0`; `c = 5`,求 `a and b or c` 的值,并解释求值顺序。

✓ 参考答案

结果为 `5`。`and` 优先于 `or`: `a and b` 得 `0` (`a` 真则继续,返回 `b`), `0 or c` 得 `5`。`(a and b) or c` 结果不变;复杂布尔表达式必须靠括号控制。

Q 3. 为什么 `x == None` 应该写成 `x is None`? 举一个 `==` 会给出错误结果的例子。

✓ 参考答案

`None` 是单例,"是否为 `None`"是身份问题, `is` 的语义更精确;且自定义类型可能重载 `__eq__` 把 `x == None` 变成任意结果。例子:定义一个 `__eq__` 恒返回 `True` 的类,则 `obj == None` 为 `True` 而 `obj is None` 为 `False`。PEP 8 明确要求判 `None` 用 `is`。

Q 4. 用一行表达式把字符串 `"hello world"` 中的元音字母收集成列表(提示:列表推导 + `in`)。

✓ 参考答案

`[ch for ch in "hello world" if ch in "aeiou"]` → `['e', 'o', 'o']`。要点: `ch in "aeiou"` 是字符串成员判断($O(n)$ 但 n 只有 5);生产可用 `set("aeiou")` 提速。

Q 5. 写出与 C 代码 `result = (x > 0) ? x * 2 : -x;` 等价的 Python 表达式。

✅ 参考答案

`result = x * 2 if x > 0 else -x`。注意"真分支"写在 `if` 前;它是表达式,可用于赋值、传参、推导式。

Q 6. 下面的代码有什么问题? `count = get_count() or 0`,其中 `get_count()` 返回数据库记录条数。

✅ 参考答案

当业务要求区分"查询失败(None)"与"确实 0 条"时, `None or 0` 把两种情况合并了。正确: `c = get_count(); count = 0 if c is None else c`。教训:假值与 `None` 是不同的语义。

章末实验题:表达式求值计算器

实验 2:表达式求值计算器

实验目标:实现一个命令行计算器,读入"数字 运算符 数字"表达式,支持 `+` `-` `*` `/` `//` `%` `**`,正确处理负数、优先级(幂高于乘除高于加减)、除零与非法输入,并把 `and` `or` `is` `in` `walrus`/条件表达式等本章知识点融进实现。

实验要求:

- 任务 1:读入形如 `7 / 2`、`-7 // 2`、`2 ** 10`、`17 % 5` 的三段式表达式并正确求值(知识点:真除法/整除/取模/幂、负数语义、`strip` 清理)
- 任务 2:用 `in` 校验运算符合法性,非法运算符提示后重新输入(知识点:成员判断、短路)
- 任务 3:对 `//` 与 `%` 的除数为 0 给出友好报错;对非数字输入用 `float()` 转换并捕获 `ValueError` (知识点:类型转换、异常捕获)
- 任务 4:用条件表达式区分"结果是否为整数",整数显示为整、浮点保留 4 位小数(知识点:三目、格式化)
- 任务 5:统计已计算次数与最近一次结果,用 `walrus` 在循环条件里读取输入;支持 `quit` 退出(知识点:`walrus`、链式比较、`while`)
- 任务 6:每次运算后打印一行历史摘要,格式: [第N次] `7 / 2 = 3.5` (知识点:f-string 与解构)

实验环境:Python 3.10+,单文件 `calc.py` 即可,命令行运行。

第一步 循环先写骨架: `while (expr := input(...)) != "quit"`,体会 `walrus` 消除重复输入调用

第二步 析用 `expr.split()` 得到三个部分,长度不对时提示"格式:数字 运算符 数字"

第三步 算分发用 `dict` 映射运算符到 `lambda`(第4章会细讲函数与 `dict` 的结合,这里先用 `if/elif` 链即可)

第四步 出统一走一个 `fmt_result` 函数,内部用条件表达式判断整数显示

✅ 参考答案要点(代码分两段)

第一段:运算符分发表与解析。分发表用 `dict + lambda` ——这是 Python 替代 C 的 `switch` 语句的惯用法;解析与计算分离,非法输入在解析层被过滤。

```
# calc.py — 表达式求值计算器(覆盖第2章全部知识点)
import math

# 运算符分发表:键是运算符,值是"两个数的计算函数"
OPS = {
    "+": lambda a, b: a + b,
    "-": lambda a, b: a - b,
    "*": lambda a, b: a * b,
    "/": lambda a, b: a / b,          # 真除法,永远 float
    "//": lambda a, b: a // b,       # 向下取整
    "%": lambda a, b: a % b,         # 模:符号随除数
    "**": lambda a, b: a ** b,        # 幂
}

def parse_expr(line: str):
    """解析 '数字 运算符 数字' 为 (a, op, b);非法返回 None"""
    parts = line.split()
    if len(parts) != 3:
        print("格式应为: 数字 运算符 数字(如 7 / 2),运算符支持:",
              " ".join(OPS))
        return None
    a_str, op, b_str = parts          # 解构赋值
    if op not in OPS:                  # in 成员判断
        print(f"不支持的运算符: {op!r}")
        return None
    try:
        a = float(a_str)              # 类型转换
        b = float(b_str)
    except ValueError:
        print("请输入合法数字")
        return None
    return a, op, b

def fmt_result(value) -> str:
    """整数显示为整,浮点保留 4 位(条件表达式)"""
    return f"{value:.0f}" if value == int(value) else f"{value:.4f}"
```

第二段:主循环。walrus 在循环条件里完成读入与判断;链式比较做结果区间提示;短路保护除零。

```

def main() -> None:
    print("表达式计算器:输入 '数字 运算符 数字',quit 退出")
    count = 0
    last = None                                # 最近一次结果
    # walrus:循环条件里完成读入与判断,不再写两遍 input()
    while (line := input("> ").strip()) != "quit":
        parsed = parse_expr(line)
        if parsed is None:
            continue                            # 非法输入,重新来
        a, op, b = parsed
        if op in ("/", "%") and b == 0:         # 短路保护除法
            print("除数不能为 0")
            continue
        result = OPS[op](a, b)                  # 分发执行
        count += 1
        last = result
        if (a < 0 or b < 0) and op in ("/", "%"):
            print(f"    [提示] 负数{op}:向下取整,余数符号随除数")
        print(f"[第{count}次] {a:g} {op} {b:g} = {fmt_result(result)}")
        # 链式比较:结果范围提示
        print("    结果在 [0, 100) 区间内" if 0 <= result < 100
              else "    结果超出 [0, 100) 区间")
    if last is not None:
        ok = math.isclose(last, float(int(last)))
        print(f"共计算 {count} 次,最后结果 {last}(整数?{ok})")

if __name__ == "__main__":
    main()

```

设计说明:① 运算符分发表 OPS 让"加一个新运算符 = 加一行",这是 C 的 switch 在 Python 里的正统替代;② 解析与计算分离,非法输入在解析层被过滤,计算层只需专注数学;③ 负数整除提示语帮助学生现场巩固 `//` / `%` 的符号规则;④ 全部输出走 `fmt_result`,保证"3.0"显示成"3"——这是生产日志里最常见的格式化细节。

下一章预告:第3章 流程控制——`if` 的真值判定、`match` 结构匹配、`for` 遍历万物、`for-else` 与 `while-else` 的奇技、没有 `goto` 的世界。